

A Corrida Armamentista: Analistas vs. Criadores de Malware

A análise de malware não é um processo estático, mas sim uma evolução contínua. Este repositório documenta o estudo de técnicas de **Engenharia Reversa** aplicadas para desvendar as defesas criadas por autores de ameaças.

1. O Conceito de Engenharia Reversa

Em cibersegurança, a engenharia reversa é o processo de dissecar um artefato tecnológico (software ou hardware) para:

- **Extrair IOCs:** Identificar IPs, domínios e hashes de arquivos maliciosos.
- **Mapear Funcionalidades:** Entender o que o malware faz (ex: roubo de credenciais, criptografia de dados).
- **Desenvolver Contramedidas:** Criar vacinas, assinaturas de antivírus e regras de firewall.

2. A "Corrida Armamentista" (Armored Malware)

Este termo descreve o ciclo de escalada entre dois lados:

- **Autores de Malware:** Motivados a proteger seus programas, utilizam técnicas **Anti-Engenharia Reversa** para retardar a análise e maximizar o tempo de infecção.
- **Analistas de Segurança:** Desenvolvem novos métodos e ferramentas para contornar essas proteções e mitigar as ameaças antes que se propaguem.

"À medida que cada lado aprimora suas técnicas, o outro responde com outras ainda mais avançadas, criando um ciclo de escalada contínuo."

Defesas de Software: Técnicas Anti-Depuração

A depuração interativa é a maior ameaça ao criador de malware, pois permite ao analista entender a lógica do ataque. Para combater isso, os desenvolvedores de ameaças implementam proteções ativas.

1. Detecção de Ambiente (IsDebuggerPresent)

A técnica mais comum envolve o uso de APIs nativas do Windows. O malware interage com o Sistema Operacional para descobrir se está sendo monitorado.

- **IsDebuggerPresent:** Verifica um sinalizador no bloco de ambiente do processo (PEB). Se retornar "True", o malware pode simplesmente se fechar.

- **CheckRemoteDebuggerPresent:** Semelhante à anterior, mas pode verificar processos vizinhos.

2. Adulteração de Registradores de Hardware

Os depuradores usam registros especiais da CPU (como DR0, DR1, DR2, DR3) para definir **Hardware Breakpoints**.

- O malware executa códigos que tentam redefinir esses registros. Se ele conseguir limpá-los, o depurador perde a capacidade de pausar a execução em pontos críticos, tornando a análise cega.

3. Código Automodificável (Self-Modifying Code)

Nesta técnica, o malware é carregado em um estado "dormente" ou ofuscado.

- Durante a execução, ele escreve novas instruções sobre as antigas na memória RAM.
- **Impacto:** Isso quebra a análise estática (o que você vê no Ghidra não é o que roda) e dificulta a depuração, pois o código muda enquanto você o percorre.

4. Suspensão de Threads (SuspendThread)

Vimos na prática que o malware pode enumerar os processos do sistema e, ao encontrar o depurador, chamar a API `SuspendThread`.

- Isso congela a ferramenta do analista, impedindo qualquer avanço no código e forçando a reinicialização da análise — um ciclo exaustivo para o defensor.

Estudo de Caso: Neutralizando a API `SuspendThread`

Nesta análise, investigamos um binário que utiliza uma técnica ativa de anti-depuração para paralisar o analista. O malware ataca o próprio depurador para impedir a investigação.

1. Mecanismo de Ataque (Análise de Código)

O malware `suspend-thread.exe` opera seguindo esta lógica:

1. **Enumeração de Janelas:** Utiliza a função `EnumWindows` para listar todas as janelas abertas no sistema.

2. **Identificação por String:** Busca por títulos que contenham termos como "dbg", "debugger" ou "debug".
3. **Ataque Direto:** Ao encontrar o PID (ID do Processo) do depurador, ele percorre as threads desse processo e chama a API SuspendThread.
4. **Resultado:** O **x64dbg** para de responder (trava), impossibilitando a continuidade da análise.

2. Técnica de Defesa: Patching de Chamada Intermodular

Para vencer essa defesa, aplicamos um **patch** diretamente na chamada da API maliciosa.

Passo a Passo da Solução:

1. **Localização:** No x32dbg, usamos a busca por "**Intermodular Calls**" para encontrar exatamente onde o programa solicita ao Windows a função SuspendThread.
2. **Neutralização (NOPing):** Substituímos a instrução CALL <JMP.&SuspendThread> por instruções NOP (0x90).
 - *Nota:* Como a instrução original ocupava 6 bytes, o depurador preencheu o espaço com 6 NOPs para manter a integridade do binário.
3. **Verificação:** Ao executar o programa modificado, o console exibe a mensagem de que o depurador foi encontrado, mas a execução **não trava**, provando que a "arma" do malware foi removida.

3. Gestão de Patches (Export & Import)

Como as alterações na memória do depurador são voláteis (desaparecem ao reiniciar o processo), utilizamos o recurso de **Patch File**:

- **Exportar:** Salva as modificações de endereços de memória em um arquivo externo.
- **Importar:** Permite que, em uma nova sessão de análise, o analista aplique instantaneamente todas as modificações anteriores sem precisar localizar as instruções novamente.

Detecção de VM: Quando o Malware Sabe que Está Sendo Observado

Uma das primeiras coisas que um malware faz ao "acordar" em um sistema é olhar ao redor para ver se está em uma **Sandbox** ou **Máquina Virtual (VM)**. Se ele perceber que o ambiente é controlado, ele simplesmente não mostra sua face real.

O Comportamento Camaleão

Se o malware detecta uma VM, ele pode:

- **Ficar inerte:** Não executa nenhuma ação maliciosa (o analista acha que o arquivo é inofensivo).
- **Autodestruição:** Deleta o próprio executável para não deixar rastros.
- **Carga Falsa:** Abre a calculadora ou um site legítimo para enganar o monitoramento.

Como ele nos encontra? (Técnicas de Fingerprinting)

Eu aprendi que o malware busca por "impressões digitais" que nós, analistas, costumamos deixar passar:

1. Rastros de Software e Processos

O malware usa a API `EnumProcesses` para buscar nomes óbvios como `vmtoolsd.exe` (VMWare) ou `VBoxService.exe` (VirtualBox). Ele também olha o registro em `Uninstall` para ver se ferramentas como o **Ghidra** ou **Wireshark** estão instaladas.

2. O Endereço MAC (O "RG" da Placa de Rede)

As VMs geram endereços MAC com prefixos específicos (OUIs). Se o malware vê algo começando com `08:00:27` (VirtualBox) ou `00:05:69` (VMWare), ele sabe na hora onde está.

3. Escassez de Recursos

Sandboxes automáticas geralmente são configuradas com o mínimo:

- **RAM:** Menos de 4GB.
- **CPU:** Apenas 1 núcleo.
- **Disco:** Menos de 60GB. Malwares modernos consideram esses números "suspeitos" e param a execução.

4. Falta de Interação Humana

Uma das técnicas mais inteligentes é verificar periféricos. Uma máquina real quase sempre tem uma **Impressora** instalada ou um histórico de documentos abertos. Uma VM de análise geralmente está "limpa" demais.

Estratégia Anti-VM

Para contornar isso, não podemos apenas "torcer" para não sermos vistos. Precisamos de **Cloaking** (Camuflagem):

- **Scripts de Automação:** Ferramentas como VMwareC1oak ou VBoxC1oak ajudam a limpar o registro e renomear drivers.
- **Hardening da VM:** Eu configuro minhas VMs para reportarem mais RAM e núcleos de CPU do que realmente usam, e instalo drivers de impressora genéricos para parecer uma máquina de escritório real.
- **Patching:** Se eu encontrar a função de verificação de VM no depurador, eu simplesmente aplico um **NOP** ou mudo o salto (como fiz na tarefa anterior), forçando o malware a ignorar que está em uma VM.

Análise de Técnicas de Detecção de Ambiente (Anti-VM)

A detecção de Máquinas Virtuais (VMs) é uma das defesas mais comuns implementadas em artefatos maliciosos. O objetivo é identificar se o ambiente de execução é controlado (Sandbox ou laboratório de análise) para mitigar a exposição de suas funcionalidades reais.

1. Impacto da Detecção no Fluxo de Execução

Ao confirmar a presença de uma VM, o malware pode alterar seu comportamento para evadir a análise:

- **Execução Mínima:** Oculta módulos críticos e executa apenas funções benignas.
- **Mecanismo de Autodestruição:** Exclusão do binário ou corrupção do próprio código em disco.
- **Carga Ofensiva:** Ataque direto ao sistema host ou criptografia de arquivos como medida punitiva.
- **Terminação Silenciosa:** O processo é encerrado sem realizar qualquer atividade.

2. Metodologias de Identificação de Artefatos

O malware utiliza a combinação de múltiplas técnicas para validar o ambiente de execução:

A. Enumeração de Processos e Serviços

Através da API EnumProcesses, o código busca por nomes de processos vinculados a hipervisores conhecidos, como:

- **VMware:** vmtoolsd.exe

- **VirtualBox:** VBoxService.exe

B. Inspeção do Registro do Windows

Busca por chaves de instalação e configuração de ferramentas forenses e drivers de virtualização no caminho:

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall

11

C. Fingerprinting de Rede (OUI)

Análise do endereço MAC da interface de rede. Prefixos (OUI) específicos são atribuídos a fornecedores de virtualização:

- **VirtualBox:** 08:00:27
- **VMware:** 00:05:69, 00:0c:29, 00:1c:14, 00:50:56

D. Verificação de Recursos de Hardware

Sistemas de análise automatizada frequentemente operam com recursos limitados. O malware valida:

- **Capacidade de RAM:** Máquinas com menos de 4GB são tratadas como suspeitas.
- **Núcleos de CPU:** Verificação de sistemas mono-processados.
- **Periféricos:** Ausência de drivers de impressão ou dispositivos USB genéricos.

E. Análise de Atividade do Usuário

Verificação de variáveis de ambiente para confirmar se a máquina faz parte de um domínio (Active Directory) através das variáveis ComputerName e LogonServer, além de monitorar a ausência de histórico de navegação ou arquivos recentes.

3. Estratégias de Mitigação (Anti-VM Cloaking)

Para neutralizar essas detecções, aplicamos técnicas de endurecimento (*hardening*) no ambiente de análise:

1. **Ofuscação de Artefatos:** Uso de scripts (ex: VMWareCloak) para remover entradas de registro e renomear drivers de sistema.
2. **Simulação de Hardware:** Configuração manual do hipervisor para reportar endereços MAC genéricos e alocação de recursos condizentes com máquinas físicas.
3. **Patching Dinâmico:** Identificação das rotinas de verificação no depurador e manipulação dos saltos condicionais para forçar o caminho de execução pretendido.

Conclusão Técnica

A detecção de VM não é infalível, mas representa um obstáculo significativo. A eficácia da análise dinâmica depende da capacidade do analista em mimetizar um ambiente de usuário final real, eliminando qualquer rastro que denuncie a natureza virtualizada do laboratório.

Detecção de VM via Sensores de Temperatura (WMI/ACPI)

Enquanto muitas técnicas de Anti-VM procuram por nomes de drivers ou processos, esta técnica interroga diretamente a infraestrutura de gestão do Windows (**WMI - Windows Management Instrumentation**) para verificar a existência de dados térmicos reais.

1. Fundamentos da Técnica

Em sistemas físicos, a classe Win32_TemperatureProbe ou a query WQL MSAcpi_ThermalZoneTemperature acedem a dados do **SMBIOS**.

- **Em Máquinas Físicas:** Retornam a temperatura atual dos componentes (CPU, Board).
- **Em Máquinas Virtuais:** Geralmente, o hardware simulado não fornece leituras térmicas reais, retornando o erro Not Supported ou valores nulos.

O malware interpreta a ausência de dados térmicos como uma confirmação de que está a correr num ambiente virtualizado e interrompe a sua carga maliciosa para evitar a análise.

2. Metodologia de Bypass: Manipulação de Memória e Registos

Demonstrámos que, mesmo quando o código está compilado para detetar a VM, o analista pode intervir em tempo de execução utilizando o **x32dbg** através de dois métodos avançados:

A. Manipulação Direta de Memória (Memory Patching)

Em vez de alterar o código (instruções), alteramos os **dados** que o código processa:

1. Localizámos o endereço de memória da variável de retorno (uReturn em [ebp-18]).
2. O malware recebeu 0 (Falha/VM detectada).
3. Através da janela de **Dump**, alterámos manualmente o bit para 1 (Sucesso/Físico).
4. **Resultado:** A instrução de comparação (CMP) avaliou o valor modificado, enganando a lógica do malware.

B. Alteração do Fluxo via Registo EIP (Instruction Pointer)

O registo **EIP** (Extended Instruction Pointer) é o "ponteiro" que diz à CPU qual é a próxima instrução a executar.

1. Identificámos o endereço de memória da rotina maliciosa que queríamos alcançar (ex: o printf de sucesso).
2. Modificámos o valor do registo **EIP** diretamente no painel de Registos do depurador.
3. **Resultado:** "Teleportámos" a execução do programa, saltando completamente por cima de todas as verificações de segurança e indo direto para a carga útil.

3. Comparativo de Técnicas de Bypass

Técnica	Aplicação	Vantagem
NOPing	Alteração do Código	Permanente e simples de aplicar em saltos.
Memory Dump	Alteração de Dados	Útil quando o código depende de variáveis complexas ou resultados de APIs.
EIP Jump	Alteração de Fluxo	A forma mais rápida de forçar o programa a executar uma função específica, ignorando proteções.

Conclusão Técnica

A detecção por temperatura demonstra a sofisticação da "corrida armamentista". No entanto, como analistas, o controlo que detemos sobre os **Registos da CPU (EIP)** e o **Endereçamento de Memória** permite-nos reescrever a realidade do programa enquanto ele executa.

Dominar a manipulação do **EIP** é uma competência crítica, pois permite isolar funções maliciosas específicas sem ter de resolver todos os "puzzles" de evasão colocados pelo autor do malware.

Ofuscação e Técnicas de Empacotamento (Packing)

A ofuscação é a arte de tornar o código e os dados deliberadamente confusos para humanos e ferramentas de segurança, sem alterar a sua funcionalidade final. Para um autor de malware, isso é essencial para evitar a detecção por assinaturas e atrasar a engenharia reversa.

1. Métodos Comuns de Ofuscação

- **Codificação (Encoding):** Uso de algoritmos como **Base64** ou **XOR** para esconder strings de comando, domínios de C2 e caminhos de arquivos.
- **Criptografia:** Proteção de módulos críticos e comunicações de rede usando chaves simétricas ou públicas, tornando os dados ilegíveis sem a chave correta.
- **Ofuscação de Código:** Alteração da estrutura lógica, renomeação de funções para nomes genéricos e inserção de "código morto" (instruções que não fazem nada) para confundir o fluxo do programa.

2. O Papel dos Packers (Empacotadores)

Um **Packer** funciona como um invólucro (*wrapper*) que comprime e, frequentemente, criptografa o executável original.

Anatomia de um arquivo empacotado:

1. **O Stub de Descompactação:** Um pequeno trecho de código que é executado primeiro.
2. **O Payload:** O malware original comprimido/criptografado.
3. **Execução em Memória:** O *Stub* descompacta o *Payload* diretamente na memória RAM e transfere o controle para ele (técnica conhecida como *RunPE* ou *Process Hollowing*).

Por que os autores de malware usam Packers?

- **Evasão de Antivírus:** Como o código malicioso está comprimido, as assinaturas estáticas de AV não conseguem encontrá-lo.
- **Redução de Tamanho:** Facilita a distribuição via e-mail ou downloads rápidos.
- **Anti-Análise:** Muitos packers modernos (como **Themida** ou **VMPProtect**) incluem proteções complexas contra depuradores e máquinas virtuais.

3. Exemplos de Ferramentas e Casos de Uso

Embora ferramentas como o **UPX** sejam legítimas e usadas para otimização, outras são focadas em proteção pesada:

- **Legítimo:** **Themida** (muito usado em sistemas Anti-Cheat de jogos) e **Dotfuscator**.
- **Malicioso/Dual-use:** **MPress**, **ASPack** e versões modificadas do **UPX**.

Desafio para o Analista

Ao realizar a análise estática de um arquivo empacotado:

- As strings legíveis serão mínimas ou inexistentes.

- A Tabela de Importações (IAT) mostrará apenas funções básicas como `LoadLibrary` e `GetProcAddress`.
- A entropia do arquivo será muito alta (indicando dados comprimidos ou criptografados).

Conclusão Técnica

O uso de um Packer não confirma que um arquivo é malicioso, mas é um forte indicador de que o software deseja ocultar a sua funcionalidade original.

Desconstruindo o Malware: Identificação e Unpacking Manual

Quando um malware utiliza um **Packer**, ele esconde sua verdadeira estrutura. O objetivo desta etapa é identificar a tecnologia de empacotamento, forçar a descompactação em memória e extrair um executável limpo para análise.

1. Identificação de Amostras Empacotadas

Antes de depurar, utilizamos ferramentas de análise estática para confirmar a presença de um Packer:

- **Detect It Easy (DIE)**: Utiliza assinaturas para identificar o Packer. A alta **Entropia** (desordem nos dados) em seções específicas é um indicador primário de compressão ou criptografia.
- **PEStudio**: Revela anomalias nas seções do arquivo PE. Nomes de seções não padronizados (ex: `UPX0`, `UPX1`) são evidências claras de manipulação por empacotadores.

2. O Processo de Unpacking Manual

A técnica mais eficaz para lidar com packers complexos é o **Dynamic Unpacking**, que consiste em deixar o próprio malware realizar o trabalho de descompactação na RAM.

Procedimento no x32dbg:

1. **Localização do OEP (Original Entry Point)**: O malware empacotado executa primeiro o código do *Stub*. Devemos encontrar o ponto onde o *Stub* termina e o código original começa.
2. **Breakpoints Estratégicos**: Definimos pontos de interrupção logo após a rotina de descompressão (ex: no salto final para o OEP).
3. **Transição de Execução**: Ao atingir o OEP, o malware está totalmente "aberto" na memória, pronto para ser extraído.

3. Dumping e Reconstrução da IAT (Scylla)

Um simples "print" da memória não é suficiente para criar um executável funcional, pois as conexões com as bibliotecas do Windows (DLLs) estão quebradas. Utilizamos o plugin **Scylla** para resolver isso:

- **Dump:** Grava o estado atual da memória do processo em um novo arquivo .exe.
- **IAT Autosearch & Get Imports:** Localiza a **Import Address Table** (Tabela de Endereços de Importação). Esta tabela é vital para que o programa saiba como chamar funções do sistema (como `CreateFile` ou `InternetConnect`).
- **Fix Dump:** Reconstrói o cabeçalho do arquivo extraído, vinculando as importações corrigidas ao novo binário.

4. Validação e Conclusão

Após o *Fix Dump*, o novo arquivo (ex: `packed_SCY.exe`) é testado. Se a descompactação foi bem-sucedida:

1. O arquivo executará normalmente fora do depurador.
2. Ferramentas como **Detect It Easy** não mostrarão mais o Packer, revelando a linguagem de programação original (ex: C++, Delphi).
3. As strings e funções maliciosas estarão agora visíveis para análise estática avançada.

Nota Técnica: O UPX é um ponto de partida didático. Packers comerciais e customizados frequentemente utilizam técnicas de **Anti-Dumping** e **IAT Obfuscation**, exigindo que o analista limpe manualmente a tabela de importações antes da reconstrução final.